Maynooth University

Dept. Electronic Engineering

EE695

SIMULATION OF LOW EARTH ORBIT CONSTELLATIONS IN RESPECT TO RURAL BROADBAND COVERAGE IN IRELAND

A Project Report submitted in partial fulfilment of the requirements of the Maynooth University for the degree of Masters of Engineering

Author:
Alan Farrell

Student Number:
21329721

Facilitator
DR Mustafa Kishk

April 21, 2026

Contents

1 Introduction **1**

2 Literature Review **2**

3 System Design and Methodology **3**

3.1 Data Inputs . . . 3

3.2 Two Line Element Data . . . 3

3.3 Orbit Propagation . . . 4

3.4 Coordinate Transformations . . . 4

3.4.1 TEME . . . 4

3.4.2 ECEF . . . 4

3.5 Satellite Visibility Model . . . 4

3.6 Beam-width Model . . . 5

3.7 Coverage Window Detection . . . 5

4 Implementation **6**

4.1 Simulation Parameter Initialization . . . 6

4.2 Data acquisition . . . 6

4.3 Orbit propagation . . . 8

4.4 Coordinate Transformations . . . 9

4.4.1 TEME to ECEF . . . 9

4.4.2 Geodetic ECEF . . . 10

4.5 Visibility Logic . . . 11

4.6 Beam width Coverage Model . . . 11

4.7 Coverage Window Detection and Tracking . . . 11

4.8 Coverage Capability . . . 12

4.9 Optimal Satellite Selection . . . 13

5 Results **14**

5.1 5G Coverage Analysis . . . 14

5.2 Performance Metrics . . . 14

5.2.1 Throughput . . . 14

5.2.2 Latency . . . 14

5.3 Conclusions . . . 14

Abstract

1. Introduction

INTERIM REPORT CONTENT

Background & motivation for LEO satellite constellations

Why coverage analysis matters (global broadband, latency, constellation optimization)

Limitations of GEO/MEO → why LEO

Overview of SGP4 propagation, visibility checks, coverage model

Project aims & objectives

Scope of the project

2. Literature Review

INTERIM REPORT CONTENT

Overview of LEO constellations (Starlink, OneWeb, Kuiper)

Use cases for my project in other countries

3. System Design and Methodology

3.1 Data Inputs

The main data input to the propagation of satellites in this project is Two-Line Element sets (TLEs). TLEs are a common publicly accessible format which describes Earth-orbiting satellite positions. The primary source of TLE data is SpaceTrack [1]. Although SpaceTrack is the primary source, many projects such as this project and other research rely on sources such as CelesTrak, which takes the data from SpaceTrack and organizes it into specific groupings [2]. Some examples of these groupings are shown in Listing 3.1.

- **Starlink**
- **ISS**
- **Earth observation satellites**
- **Weather satellites**
- **Communications satellites**
- **Active vs. debris groupings**

Figure 3.1: Common satellite groupings provided by CelesTrak

While other sites exist such as N2Y0 and AMSAT, CelesTrak remains one of the most reliable and widely used platforms due to consistent updates synchronized with SpaceTrack and curated categories which help simplify data access. CelesTrak updates the majority of its TLE files approximately every two hours. Its Starlink data can expect multiple updates per day, making it suitable for this project's application [3].

3.2 Two Line Element Data

TLEs are a standardized format used to describe the orbital properties of Earth-orbiting satellites. A TLE represents a satellite's orbit at a specific moment in time, known as the epoch. Because the TLE format is simple and lightweight, it is widely supported and remains the most common method for distributing orbit data for tracking and prediction.

A TLE consists of three lines: line 0 (optional) contains the satellite name, line 1 contains identification and basic orbital parameters, and line 2 describes remaining orbital elements such as inclination. This is summarized in Table 3.1. An example of TLE data is shown in Listing 3.2.

```
STARLINK-5694
1 55481U 23015AJ  26041.24424508  .00002920  00000+0  23114-3 0  9992
2 55481  42.9978 128.2652 0001315 248.0292 112.0411 15.02536911166474
```

Figure 3.2: Two-Line Element (TLE) for STARLINK-5694

Table 3.1: Summary of key parameters in a Two-Line Element

Field	**Value**	**Meaning**
Satellite Name	STARLINK-5694	Human-readable satellite identifier
Catalog Number	55481	Unique NORAD satellite ID
Classification	U	Unclassified satellite
Launch Year	2023	Part of the International Designator (23)
Launch Number	015	15th launch of the year 2023
Launch Piece	AJ	Specific object from that launch
Epoch (Year)	2026	TLE epoch year (26)
Epoch (Day)	41.24424508	Day of year when the TLE is valid
Inclination (deg)	42.9978	Tilt of orbit relative to Earth's equator
Mean Motion (rev/day)	15.02536911	Orbits per day (≈95.8 min period)

3.3 Orbit Propagation

The Simplified General Perturbations Model 4 (SGP4) [4] is used for orbit propagation.

3.4 Coordinate Transformations

Coverage estimation first requires the satellite and observer to be expressed in a common coordinate system at a given epoch. Satellites propagated using SGP4 are expressed in the True Equator Mean Equinox (TEME) frame, while ground sites are typically given in geodetic coordinates. It is therefore necessary to convert both to an Earth-Centered Earth-Fixed (ECEF) frame [5, 6].

3.4.1 TEME

TEME is the native output frame of SGP4. It is non-rotating with respect to Earth's surface and must be converted before comparison with geodetic locations [5].

3.4.2 ECEF

ECEF is a rotating Cartesian frame whose origin is at Earth's center of mass. The axes align with Earth's rotation and geometry [7]. This is shown in Figure 3.3.

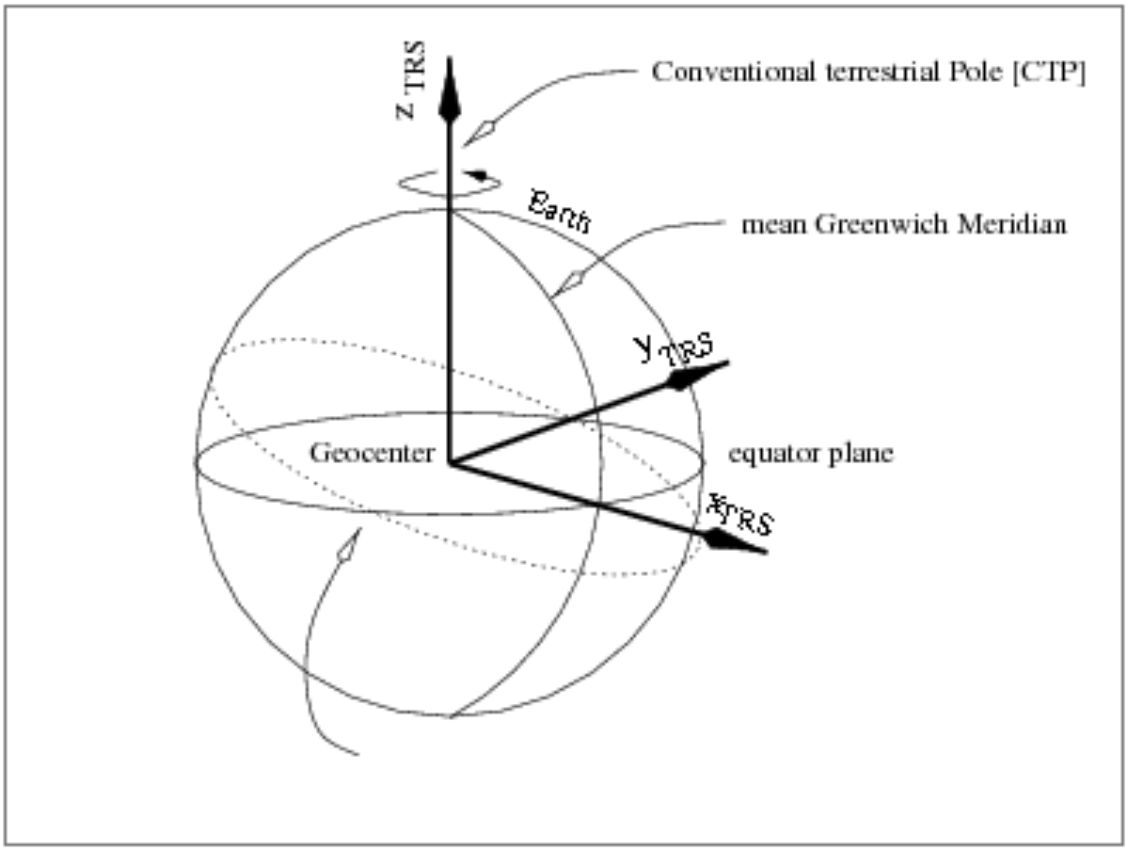

Figure 3.3: ECEF Coordinate System

3.5 Satellite Visibility Model

Visibility is evaluated using line-of-sight (LOS) analysis. The vector from observer to satellite is

$$d = r_{sat} - r_{obs}. \tag{3.1}$$

The LOS vector is transformed into the East-North-Up (ENU) frame, enabling calculation of the elevation angle [8].

$$\theta_{el} = \arctan\left(\frac{u}{\sqrt{e^2 + n^2}}\right) \tag{3.2}$$

Only satellites exceeding a 15° elevation mask are considered visible [9].

3.6 Beam-width Model

Visibility alone does not guarantee service. A simplified nadir-pointing beam model is used, where a user must lie within a maximum off-nadir angle θ_{beam}. Only satellites satisfying both elevation and beam-width constraints are deemed capable of providing coverage.

3.7 Coverage Window Detection

Coverage windows are determined by evaluating satellite visibility at discrete time steps. A window begins when coverage becomes available and ends when it is lost. For each window, the system records start time, end time, duration, and optimal satellite selection.

4. Implementation

This chapter describes the logic behind the software implementation of the satellite coverage and capacity analysis which is detailed in Chapter 3. The system is implemented in Python and is designed to evaluate the availability and theoretical throughput of LEO satellite constellations for rural regions in Ireland.

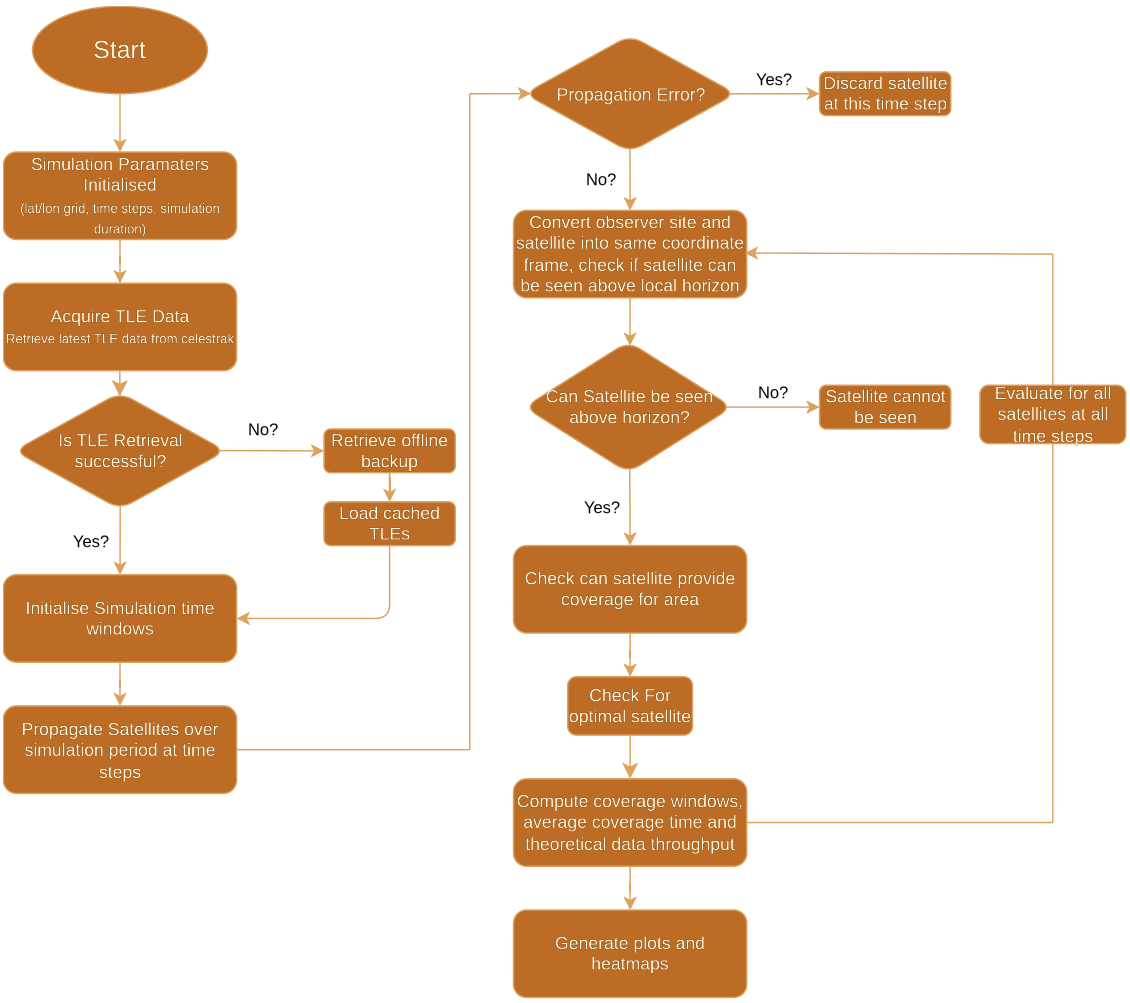

Figure 4.1: Implementation Logic

Figure 4.1 illustrates the overall execution and flow of the implementation logic. The implementation begins by setting the simulation Parameters, i.e, duration of simulation, frequency of time steps, the density of the coverage grid, and the size of the area being studied. Following this, the TLE data is then acquired from CelesTrak, which is used to propagate satellite orbits using the SGP4 model. At each discrete time step, satellite positions are propagated and then transformed into a common Earth fixed coordinate frame, and evaluated using an elevation mask and beam width constraints to determine which satellites can provide coverage and when.

The core execution of this system is based in a loop which iterates over a grid of latitude and longitude coordinated within which visibility and coverage check are preformed for all propagated satellites at each time step. After the completion of this simulation, the collected data is stored and then processed into heatmaps to be visualized, this data will then be compared tio publicly available terrestrial data.

4.1 Simulation Parameter Initialization

4.2 Data acquisition

The first stage of implementation is acquiring up to date orbital Parameters for the Starlink LEO constellation. TLE sets are retrieved from Celestrak. This data provides the perturbations required to propagate the orbits using SGP4. The TLE data used in this system are taken directly from Celestrak [2]. Downloading this data produces a text stream which has sets of three lines of text blocks, satellite name, followed by line 1 and line 2 of the TLE. The implementation of this system parses these entries for each satellite.

Using the most up to date TLEs ensures that the system accurately represents the dynamic nature of large LEO constellations, which orbital parameters may change drastically and regularly due to factors such as manual maneuvers.

Acquiring TLE data can be done through a link from CelesTrak. This project downloads raw TLE from CelesTrak using an HTTP request and then parses the reponse into a list of lines for processing. Python makes this simple with a requests library, which can be seen implmented in listing 4.1

Listing 4.1: Pulling TLE data

```
response = requests.get("https://celestrak.org/NORAD/elements/gp.php?GROUP=starlink&
    FORMAT=tle")
lines = response.text.splitlines()
```

However, this can create issues, as this project is pulling TLE data from thousands of satellites over many time steps, this can create a large number of HTTP requests which can be blocked by CelesTrak. To avoid this, the TLE data is pulled once at the start of the simulation and then stored locally for the duration of the simulation. This allows for a more efficient and robust implementation, as it avoids potential issues with rate limiting and ensures that the same TLE data is used throughout the simulation for consistency.

On top of this it is critical to ensure that the TLE data is parsed at the right location in the code, as seen in figure 4.1, this project employs a 0.1 lat lon step for mapping over Ireland, which equates to 1000s of grid positions, and over a duration of 24 hours with a time step of 10 seconds, in order to prevent temporary IP bans, the TLE data must be pulled first, then stored, then used.

Even with this approach, this project experienced some rate limiting, throttling and temporary IP bans from CelesTrak, to get around this, as offline backup of the TLE data was stored and manually updated periodically. This can be seen implemented in 4.2. Should there be issues with accessing data directly from CelesTrak, it was still possible to, design and test while using an offline backup file containing TLE data.

Listing 4.2: TLE data acquisition logic

```
        try:
        if source.get("type") == "online":
            response = requests.get(source["url"], timeout=8)
            response.raise_for_status()
            lines = response.text.strip().splitlines()
        else:
            raise Exception
    except Exception:
        here = os.path.dirname(os.path.abspath(__file__))
        path = os.path.join(here, "..", source["local"])
        with open(path, "r") as f:
            lines = f.read().strip().splitlines()

    return lines
```

It is important to note that while this approach ensures consistency and avoids rate limiting, it may not capture any orbital changes that occur during the simulation period. For long simulations, spanning multiple days, it may be necessary to implement a strategy for periodically updating the TLE data to ensure that the simulation remains accurate over time.

Once TLE data is acquired, it is parsed and stored in a structured format for use in the SGP4 propagation stage. Each satellite's TLE is associated with its name and catalog number, allowing for easy reference during propagation and visibility analysis. There are two types of TLE data, the first is seen before in Listing 3.2 , with lines 0, 1, and 2, the second type of TLE data does not include line 0, and only has lines 1 and 2 and example of this is shown in Listing 4.3, which is a OneWeb satellite.

Listing 4.3: Two-Line Element (TLE) for SATELLITE-44057

```
1 44057U 19010A   26102.43981189  .00000080  00000+0  17451-3 0  9993
2 44057  87.9055 241.8469 0002002 112.4154 247.7189 13.16593523342917
```

The implementation of this project accounts for both formats, ensuring that all available TLE data can be utilized effectively, and can be seen in 4.4

Listing 4.4: TLE data acquisition logic

```
    tles = []
    i = 0
```

```
    while i < len(lines) - 1:

        # Case 1: bare 2-line TLE
        if lines[i].startswith("1 ") and lines[i + 1].startswith("2 "):
            line1 = lines[i].strip()
            line2 = lines[i + 1].strip()
            name = f"SAT-{line1[2:7]}"
            i += 2
        else:
            name = lines[i].strip()
            line1 = lines[i + 1].strip()
            line2 = lines[i + 2].strip()
            i += 3

        tles.append((name, line1, line2))

    return tles
```

This results in an array which can be easily iterated over for propagation and visibility checks, an example of this array is shown in Listing 4.5.

Listing 4.5: In-memory representation of parsed TLE data as stored in the `tles` array

```
tles = [
    (
        "SAT-44057",
        "1 44057U 19010A   26102.43981189  .00000080  00000+0  17451-3 0  9993",
        "2 44057  87.9055 241.8469 0002002 112.4154 247.7189 13.16593523342917"
    ),
    (
        "STARLINK-5694",
        "1 55481U 23015AJ  26041.24424508  .00002920  00000+0  23114-3 0  9992",
        "2 55481  42.9978 128.2652 0001315 248.0292 112.0411 15.02536911166474"
    )
]
```

4.3 Orbit propagation

Once the TLEs have been retrieved and parsed in a suitable manner, each satellite's position and velocity are computed using the SGP4 algorithm. In this system each satellites state vector can be generated as seen in Listing 4.6.

Listing 4.6: SGP4 propagation example

```
satellite = Satrec.twoline2rv(line1, line2)
e, r, v = satellite.sgp4(jd, fr)
```

The outputs of the 4.6 are as follows:

- **r**: The satellite position vector (x, y, z) expressed in the True Equator Mean Equinox (TEME) reference frame. This can be described as the satellite's position in respect to Earth's center of mass.
- **v**: The satellite velocity vector in the TEME coordinate frame. Also in respect to Earth's center of mass, this vector describes the satellite's velocity in three-dimensional space.
- **e**: An error flag returned by the propagator, where a value of zero indicates successful propagation and non-zero values correspond to specific SGP4 error conditions.

The Julian Date (jd, fr) is computed using the current UTC timestamp to ensure a precise propagation epoch. Any satellite with a non zero error flags are discarded to avoid propagating any invalid orbital data. As seen in code 4.7, this is repeated over a set period of time, with a configured time step, to generate a time series of satellite positions and velocities for later use in calculations.

Listing 4.7: SGP4 propagation

```
while currentTime < endTime:
jd, fr = GetJulianDate(currentTime)

for name, sat in satellites:
    e, r ,v = sat.sgp4(jd, fr)
    if e == 0:
        propagated[name].append({
            "time": currentTime,
            "r": r,  # position (km)
            "v": v  # velocity (km/s)
        })
currentTime += time_delta
```

The code seen in 4.7 generates a dictionary of satellite state vectors indexed by satellite name, with each entry containing a list of time stamped position and velocity vectors. An example of this is shown in Listing 4.8. This format allows for efficient and easy access to satellite positions, and their trajectories.

Listing 4.8: Structure of propagated satellite state vectors stored in a dictionary

```
propagated = {
    "SAT-44057": [
        {
            "time": datetime(2026, 4, 12, 12, 0, 0),
            "r": [-5321.84,  2154.77,  3219.65],  # km
            "v": [ 1.92, -6.48,  3.41]            # km/s
        },
        {
            "time": datetime(2026, 4, 12, 12, 1, 0),
            "r": [-5310.12,  1765.43,  3458.90],
            "v": [ 2.04, -6.51,  3.22]
        }
    ],

    "STARLINK-5694": [
        {
            "time": datetime(2026, 4, 12, 12, 0, 0),
            "r": [1034.55, -6882.21,  1421.88],
            "v": [ 7.32,   1.14,     0.98]
        }
    ]
}
```

4.4 Coordinate Transformations

Satellite positions produced by SGP4 are given in True Equator, Mean Equinox (TEME) format. TEME remains fixed to relative to the stars not to the Earth. It uses the Earths true equatorial plane at that moment in tim, but for the x-axis it uses the mean equinox which helps smooth out some of the short term wobbles in the Earths's rotation.

As TEME does not rotate with the Earth, but ground locations given in latitude and longitude do rotate with the Earth, for visibility and elevation calculations, both the satellite and the observer location need to be in the same rotating frame. For this reason TEME must be converted to Earth Centered Earth Fixed (ECEF) coordinate system [10].

4.4.1 TEME to ECEF

The conversion for TEME to ECEF is as follows ...

$$\mathbf{r}_{\text{ECEF}} = R_3(\theta_{\text{GMST}})\, \mathbf{r}_{\text{TEME}}, \tag{4.1}$$

Where:

- $R_3(\theta)$ is a rotation matrix describing a rotation about the Earth's Z–axis.

- θ_{GMST} is the Greenwich Mean Sidereal Time at the observation epoch.

This can be written explicitly as a rotation matrix as follows ...

$$R_3(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{4.2}$$

Meaning ...

$$\begin{bmatrix} x_{\text{ECEF}} \\ y_{\text{ECEF}} \\ z_{\text{ECEF}} \end{bmatrix} = \begin{bmatrix} \cos\theta_{\text{GMST}} & -\sin\theta_{\text{GMST}} & 0 \\ \sin\theta_{\text{GMST}} & \cos\theta_{\text{GMST}} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_{\text{TEME}} \\ y_{\text{TEME}} \\ z_{\text{TEME}} \end{bmatrix} \tag{4.3}$$

This aligns the satellite position with a Earth fixed rotating frame.

4.4.2 Geodetic ECEF

To compare a satellite with an observer on Earth, the observer's latitude, longitude and altitude must be expressed as cartesian X, Y, Z coordinates in the ECEF frame. As the Earth is not a perfect sphere it is approximated as a WSG-84 ellipsoid. This is defined by it's semi major axis(the equatorial size), the flattening(how much the poles are compressed) and the first eccentricity (a measure of how non spherical the ellipsoid is). This can be seen in (4.4) [11].

$$f = \frac{a-b}{a}, \tag{4.4}$$

where a is the semi-major axis and b is the semi-minor axis of the Earth. The first eccentricity is then given by

$$e^2 = 2f - f^2. \tag{4.5}$$

Given:

- Latitude: ϕ (radians)
- Longitude: λ (radians)
- Altitude: h (meters above the WGS–84 reference ellipsoid)

WGS–84 Ellipsoid Constants:

- $a = 6378137.0$ m (equatorial radius)
- $f = \dfrac{1}{298.257223563}$ (flattening)
- $e^2 = 2f - f^2$ (first eccentricity squared)

The first step is to find the distance from Earth's center to the surface at a given latitude accounting for any potential bulges at the equator. This can be seen in 4.6 [11]

$$N(\phi) = \frac{a}{\sqrt{1 - e^2 \sin^2\phi}}. \tag{4.6}$$

Once $N(\phi)$ has been found the observer location can be described in ECEF as seen in (4.7) [11].

$$X = (N(\phi) + h)\cos\phi\cos\lambda \tag{4.7}$$

$$Y = (N(\phi) + h)\cos\phi\sin\lambda \tag{4.8}$$

$$Z = \big(N(\phi)(1 - e^2) + h\big)\sin\phi \tag{4.9}$$

These equations convert the geodetic position (ϕ, λ, h) and the satellite's equation into the ECEF frame. This allows for all later geometry such as line of sight checks to be carried out.

4.5 Visibility Logic

Once both the ground observer and the satellites propagated position are represented in ECEF the first filter check for visibility can be applied. This is based on computing the satellite's elevation angle relative to the observers horizon.

Given the satellite position,

$$r_{\text{satellite}}^{\text{ECEF}} = (s_x, s_y, s_z), \tag{4.10}$$

and the observer position,

$$r_{\text{observer}}^{\text{ECEF}} = (g_x, g_y, g_z), \tag{4.11}$$

the line of sight vector (LOS) can be represented as,

$$d = r_{\text{satellite}}^{\text{ECEF}} - r_{\text{observer}}^{\text{ECEF}} = (d_x, d_y, d_z), \tag{4.12}$$

Using the observer latitude and longitude, the LOS can be converted into local horizon coordinates. This can be done following (4.13) [12]

$$\begin{aligned} e &= -\sin\lambda\, d_x + \cos\lambda\, d_y, \\ n &= -\cos\lambda \sin\phi\, d_x - \sin\lambda \sin\phi\, d_y + \cos\phi\, d_z, \\ u &= \cos\lambda \cos\phi\, d_x + \sin\lambda \cos\phi\, d_y + \sin\phi\, d_z. \end{aligned} \tag{4.13}$$

Using this, the elevation angle can then be computed as the angle between the LOS vector and the local horizon [12].

$$\theta_{\text{el}} = \arctan\left(\frac{u}{\sqrt{e^2 + n^2}}\right) \tag{4.14}$$

Then, to filter out any satellites which are too close to the horizon or below the horizon, a mask angle of 15° is applied.

$$\text{el} \geq \theta_{\text{mask}}, \qquad \theta_{\text{mask}} = 15^\circ \tag{4.15}$$

4.6 Beam width Coverage Model

At each time step, satellite and observer positions are represented in ECEF coordinate frame. If we let the satellite position be r_{sat} and the observer position be r_{obs}, and the nadir direction of the satellite is defined as a vector pointing from the satellite towards the Earth's centre be given by $-r_{sat}$. The direction from the satellite to the observer can be found with $r_{obs} - r_{sat}$. The angle between the satellite nadir direction and the observer direction can then be computed as seen in 4.16

$$\theta = \cos^{-1}\left(\frac{(-\mathbf{r}_{sat}) \cdot (\mathbf{r}_{obs} - \mathbf{r}_{sat})}{\|\mathbf{r}_{sat}\| \, \|\mathbf{r}_{obs} - \mathbf{r}_{sat}\|}\right) \tag{4.16}$$

A satellite is capable of providing server if this angle is less than or equal to the configured half power beam with of a satellite θ_{beam}.

The beam width parameter is configured within the system allowing different antenna characteristics to be evaluated without modifying the core implementation. This beam width check is executed for every visible satellite at each simulation time step. In this project the half power is set to be 7.5°, this is a liberal angle in order to attempt to account for beam forming techniques. [13]

$$\theta \leq \theta_{beam} \tag{4.17}$$

4.7 Coverage Window Detection and Tracking

To determine coverage for a given area, the system iterates over a grid of latitude and longitude points within the area of interest. For each point, at each time step, the visibility and beam width checks are applied to all propagated satellites. If one or more satellites satisfy both conditions, that grid point is marked as having coverage for that time step. The system tracks when coverage becomes available and when it is lost, allowing it to identify coverage windows. Each window is recorded with its start time, end time, duration, and the satellite(s) providing coverage during that window. This information is crucial for analyzing the availability of satellite service in the region. This logic can be seen implemented in code 4.9.

Listing 4.9: Visibility window logic

```
coverage_windows = []
in_coverage = False
window_start = None

for t, _, _, visible_satellites in visibility_by_time:

    filtered_satellites, _ = BeamFilter(
        visible_satellites, jd, fr, lat, lon,
        obs_alt=0.0, beamwidth_deg=beamwidth
    )

    coverage_available = len(filtered_satellites) > 0

    if coverage_available and not in_coverage:
        in_coverage = True
        window_start = t

    if not coverage_available and in_coverage:
        in_coverage = False
        coverage_windows.append((window_start, t))

# Close final window if coverage persists
if in_coverage:
    coverage_windows.append((window_start, visibility_by_time[-1][0]))
```

4.8 Coverage Capability

Coverage capability is measured in the theoretical maximum throughput that a satellite can provide to a given location using Shannon's capacity formula. This is calculated for each satellite that satisfies the visibility and beam width checks at each time step. The system assumes a fixed bandwidth and signal-to-noise ratio (SNR). This allows for an analysis of how much data throughput can be expected during those coverage windows.

$$P_{\text{received}}\ (\text{dBm}) = P_{\text{transmitted}}\ (\text{dBm}) + G_t\ (\text{dB}) + G_r\ (\text{dB}) - L_{\text{path}}\ (\text{dB}) - L_{\text{other}}\ (\text{dB}). \tag{4.18}$$

In Equation (4.18), P_{received} (dBm) is the power level at the receiver input after all gains and losses along the communication link have been applied. It represents the signal strength available for demodulation at the receiver.

$P_{\text{transmitted}}$ (dBm) is the output power of the transmitting radio or antenna system, expressed on a logarithmic decibel-milliwatt scale.

G_t (dB) is the transmitter antenna gain, which quantifies the ability of the transmitting antenna to direct radiated power toward the receiver relative to an isotropic radiator.

G_r (dB) is the receiver antenna gain, representing the effectiveness of the receiving antenna in capturing incoming electromagnetic energy from the direction of the transmitter.

L_{path} (dB) is theaptoror free-space path loss, accounting for signal attenuation due to geometric spreading of the electromagnetic wave as it propagates over the distance between transmitter and receiver.

L_{other} (dB) represents additional system losses not captured by free-space path loss alone, including atmospheric attenuation, polarization mismatch, pointing losses, hardware losses, and implementation-related inefficiencies.

Equation 4.18 (Link budget equation), calculates the received power, in dBm based on transmitted power, antenna gains, and losses in the communication path. This allows the power of a signal at a user's location to be calculated.

$$P_{\text{interference}} = 10\log_{10}\left(\sum_{i=1}^{N} 10^{\frac{P_i}{10}}\right). \tag{4.19}$$

Equation 4.19 [14] converts dBM into watts, sums all interfering signals received at a given location, and converts these interfering signals back into dB. This allows us to see the total interfering power.

$$P_{\text{noise}} = -174 + 10\log_{10}(B) + NF. \tag{4.20}$$

Equation 4.20 [15]calculates the thermal noise power at room temperature, which simply gives a baseline which can be used to estimate background noise which will limit how well a receiver could detect a signal. B is the channel bandwidth in Hz and NF is the noise figure which comes from the receiver hardware.

$$\mathrm{SNR} = P_{\mathrm{signal}} - P_{\mathrm{noise}}. \tag{4.21}$$

Equation 4.21 [15]calculates the signal to noise ratio, this is a measure of the main signal's strength relative to background noise.

$$\mathrm{SIR} = P_{\mathrm{signal}} - P_{\mathrm{interference}}. \tag{4.22}$$

Equation 4.22 [15] is similar to 4.20, except that it calculates the strength of the main signals relative to interference signals from nearby towers.

$$\mathrm{SINR}\ (\mathrm{dB}) = 10\log_{10}\left(\frac{10^{\frac{P_{\mathrm{signal}}}{10}}}{10^{\frac{P_{\mathrm{interference}}}{10}} + 10^{\frac{P_{\mathrm{noise}}}{10}}}\right). \tag{4.23}$$

SINR is calculated as the main signal power divided by the sum of interference and noise. This is done in a linear scale (mV) and then converted back into dB.

Given these calculations, the Shannon capacity (theoretical maximum data rate) can be found. This can be seen in 4.24 [14]

$$C = B \cdot \log_2\left(1 + \mathrm{SINR}_{\mathrm{linear}}\right). \tag{4.24}$$

These equations were implemented in the system to calculate the theoretical maximum throughput for each satellite providing coverage at each time step. This allows for an analysis of coverage in Ireland.

4.9 Optimal Satellite Selection

5. Results

The three core metrics which will be compared are coverage availability (as a percentage of time), capacity throughput (Mbps) and latency (ms).

Coverage availability determines how often the user has service, this metric is based on visibility of satellites and a high level model of the beamwidth of a given satellite. This will be compared to urban and rural 5G networks. Throughput is measured in Mbps, this is averaged over a period of time and will also be compared to urban and rural 5G throughput.

Number of satellites visible over time

Plots (elevation vs. time, count of visible satellites, etc.)

Sample day simulation

Visualisations

Coverage heatmaps

Capacity throughput estimates

5.1 5G Coverage Analysis

5.2 Performance Metrics

5.2.1 Throughput

5.2.2 Latency

5.3 Conclusions

(a) 5G coverage in Ireland – 48 [16]

(b) 5G coverage in Ireland – Eir [16]

(c) 5G coverage in Ireland – Three [16]

(d) 5G coverage in Ireland – Vodafone [16]

Figure 5.1: Network operator 5G coverage maps

Bibliography

[1] U.S. Space Force, "Space-Track.org Login," https://www.space-track.org/auth/login, 2026, accessed: 2026-02-24.

[2] CelesTrak, "CelesTrak: NORAD Two-Line Element Sets," https://celestrak.org/NORAD/elements/, 2026, accessed: 2026-02-24.

[3] USRadioGuy, "Using the New TLE Format," https://usradioguy.com/using-new-tle-format/, 2026, accessed: 2026-02-24.

[4] C. K. Ho and R. C. Johnson, "Space track: Theory and practice of satellite orbit tracking," Department of Aerospace and Ocean Engineering, Virginia Tech, Tech. Rep. AOE4134, 2014, pDF downloaded from Virginia Tech AOE archive; accessed 2025-12-05.

[5] R. Ayala, D. Ayala, D. Ruiz, and L. S. Vidal, *asteRisk: Computation of Satellite Position*, 2023, r package version 1.4.3. [Online]. Available: https://search.r-project.org/CRAN/refmans/asteRisk/html/TEMEtoITRF.html

[6] Astropy Collaboration, A. M. Price-Whelan, P. L. Lim, N. Earl, N. Starkman, L. Bradley, D. L. Shupe, A. A. Patil, L. Corrales, C. E. Brasseur, M. Nöthe, A. Donath, E. Tollerud, B. M. Morris, A. Ginsburg, E. Vaher, B. A. Weaver, J. Tocknell, W. Jamieson, M. H. van Kerkwijk, T. P. Robitaille, B. Merry, M. Bachetti, H. M. Günther, T. L. Aldcroft, and J. A. Alvarado-Montes, "The astropy project: Sustaining and growing a community-oriented open-source project," *The Astrophysical Journal*, vol. 935, no. 2, p. 167, 2022. [Online]. Available: https://docs.astropy.org/en/stable/coordinates/satellites.html

[7] J. S. Subirana, J. M. J. Zornoza, and M. Hernández-Pajares, "Conventional terrestrial reference system," 2011, navipedia, European Space Agency. [Online]. Available: https://gssc.esa.int/navipedia/index.php/Conventional_Terrestrial_Reference_System

[8] ——, "Transformations between ecef and enu coordinates," https://gssc.esa.int/navipedia/index.php/Transformations_between_ECEF_and_ENU_coordinates, 2011, navipedia, European Space Agency.

[9] A. R. Washburn, "Earth coverage by satellites in circular orbit," Naval Postgraduate School, Monterey, California, Tech. Rep., 2004. [Online]. Available: https://faculty.nps.edu/awashburn/Files/Notes/EARTHCOV.pdf

[10] D. Vallado, P. Crawford, R. Hujsak, and T. Kelso, "Revisiting spacetrack report 3," 08 2006.

[11] "Gps and ecef conversion (wgs-84 ellipsoid)," http://www.mathforengineers.com/math-calculators/GGPS-and-ECEF-converter.html, accessed 2026.

[12] J. Sanz Subirana, J. Juan Zornoza, and M. Hernández-Pajares, "Transformations between ecef and enu coordinates," https://gssc.esa.int/navipedia/index.php/Transformations_between_ECEF_and_ENU_coordinates, 2011, eSA Navipedia.

[13] C.-T. Liu and J.-Y. Pan, "Optimal beam-width for maximizing uplink coverage probability in quasi earth-fixed leo satellite communication system," *Electronics*, vol. 13, no. 7, 2024. [Online]. Available: https://www.mdpi.com/2079-9292/13/7/1349

[14] A. Goncharenko, "Optimization and analysis of wireless network characteristics with service quality maintaining," in *Proceedings of the CMiGiN 2023 Workshop, CEUR Workshop Proceedings, Vol. 3530*, 2023, pp. 24–34.

[15] European Conference of Postal and Telecommunications Administrations (CEPT), "Title to be verified," CEPT – ECO Documentation Database, Tech. Rep. Doc 1417, n.d., accessed: 2025-11-15.

[16] Commission for Communications Regulation (ComReg), "Comreg mobile coverage map," https://coveragemap.comreg.ie/map?location=56.6615732,5.410097599999999&technology=5g&network=tesco&place_id=GhIJffQ8bq5UTEARH6wQoPCjFUAbib, 2026, accessed: 2026-04-03.